OPERATING SYSTEMS

ASSIGNMENT 3 – CO3

CO3 Coverage: Evaluate memory-management mechanisms such as linking, paging and segmentation to determine their effectiveness for different system requirements – linking and loading, paging, segmentation, virtual memory, demand paging, page-replacement techniques and disk management.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment3** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assignment Principle

Every question asks you to design, implement, analyse and evaluate the solution yourself, so that you demonstrate evidence of:

Design → Implementation → Analysis → Evaluation

Q1 – Linking, Loading and Address Binding — 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Updating a Banking Application

A bank's branch software is built from many modules and libraries. When a security fix is made to the encryption library, every branch must currently receive a new 200 MB executable.

Implementation Tasks

You must:

- Build the same program with static linking and with dynamic (shared) libraries
- Compare the sizes of the executables and list the shared libraries with ldd
- Load a library at run time with dlopen() and dlsym()
- Show the addresses at which the program and its libraries are loaded (/proc/self/maps)

Evaluation

You must:

- Explain compile-time, load-time and execution-time address binding
- Explain the roles of the linker and the loader
- Evaluate static and dynamic linking for memory use, start-up time and ease of updating
- Explain position-independent code

Experimental Investigation

You must:

- Measure the start-up time and memory use of both versions
- Replace the shared library without recompiling the program and show the change
- Run two copies and show which pages are shared
- Test what happens when the shared library is missing

Conclusion

Which linking method should the bank use, and why?

Alternative Concept and Complexity Analysis

If the bank loaded the encryption library at run time with dlopen() only when needed, instead of linking it dynamically at start-up, what would the solution be? Evaluate the start-up time, the memory used when encryption is not needed, and the error handling needed if the library is missing.

Q2 – Paging — 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Memory for a Multi-User College Server

A college server runs hundreds of student programs of very different sizes. Allocating each program one contiguous block of memory left many small unusable gaps, and large programs could not be loaded although enough total memory was free.

Implementation Tasks

You must:

- Explain external fragmentation using a diagram of the contiguous allocation
- Translate logical addresses to physical addresses with a page table
- Calculate the size of the page table for a 32-bit and a 48-bit address space with 4 KB pages
- Calculate the effective memory-access time with and without a TLB

Evaluation

You must:

- Explain how paging removes external fragmentation and why internal fragmentation remains
- Evaluate page sizes of 1 KB, 4 KB and 2 MB
- Explain hierarchical (multi-level), hashed and inverted page tables
- Explain how protection and shared pages are provided with paging

Experimental Investigation

You must:

- Calculate the internal fragmentation for 100 processes of random sizes with each page size
- Calculate the effective access time for TLB hit ratios of 80%, 95% and 99%
- Find the page size of your Linux system (getconf PAGESIZE) and the page faults of a program (/usr/bin/time -v)
- Test addresses at the start and end of a page and beyond the last page

Conclusion

What page size would you choose for the college server, and why?

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>

#define PAGE_SIZE 1024
#define PAGES 8

/* Page table of a process: frame number, or -1 if the page is not in memory. */
int page_table[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1};

int main(void)
{
    unsigned logical[] = {0, 1500, 3100, 6200, 9000, 2049};
    for (int i = 0; i < 6; i++) {
        unsigned a = logical[i];
        // TODO 1: Split a into page number p = a / PAGE_SIZE and offset d = a %
        //   PAGE_SIZE. If p is outside the table print "invalid address"; if the frame
        //   is -1 print "page fault"; otherwise print the physical address frame *
        //   PAGE_SIZE + d

    }
    return 0;
}
```

Expected output:

```
    0 -> page 0 offset    0 : physical 5120
 1500 -> page 1 offset  476 : physical 2524
 3100 -> page 3 offset   28 : physical 7196
 6200 -> page 6 offset   56 : physical 3128
 9000 -> page 8 offset  808 : invalid address
 2049 -> page 2 offset    1 : page fault
```

Change PAGE_SIZE to 2048 and recalculate the results by hand before running the program again.

Alternative Concept and Complexity Analysis

If the server used an inverted page table (one entry per physical frame) instead of a per-process multi-level page table, what would the solution be? Evaluate the space used (proportional to physical memory versus to the address space of each process) and the time to translate an address (a hash look-up versus a walk of several levels).

Q3 – Segmentation **6 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Student Programs with Separate Code, Stack and Data

Students' programs have code that should be read-only and shared by everyone running the same compiler, a stack that grows, and data of varying size. Paging alone treats all of these the same.

Implementation Tasks

You must:

- Translate (segment, offset) addresses using a segment table with base and limit
- Show what happens when an offset exceeds the segment limit
- Draw how segmentation with paging (as in the Intel x86 architecture) translates an address
- Show the segments of a running program on Linux (/proc/<pid>/maps)

Evaluation

You must:

- Explain how segmentation matches the programmer's view of memory
- Explain how segments are protected and shared
- Evaluate the external fragmentation caused by segmentation
- Compare pure paging, pure segmentation and segmented paging

Experimental Investigation

You must:

- Translate 8 addresses by hand and verify them with a small program
- Trigger a segmentation fault and explain what the OS did
- Calculate the fragmentation after loading and removing 20 segments of random sizes
- Compare the size of the tables needed in each scheme

Conclusion

Why do modern operating systems rely mainly on paging?

Alternative Concept and Complexity Analysis

If the programs used pure paging instead of segmentation, what would the solution be? Evaluate the fragmentation (internal versus external), the size of the tables, the time to translate an address, and how easily code is shared and protected.

Q4 – Virtual Memory and Demand Paging — 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Running Large Medical-Imaging Software on a Laptop

A radiologist needs to run an image-processing program that uses 24 GB of memory on a laptop with 16 GB of RAM.

Implementation Tasks

You must:

- Explain the steps of handling a page fault
- Calculate the effective access time for page-fault rates of 1 in 1,000, 1 in 100,000 and 1 in 1,000,000 (memory 100 ns, fault service 8 ms)
- Explain copy-on-write and how it makes fork() fast
- Explain thrashing and the working-set model

Evaluation

You must:

- Explain how demand paging lets a 24 GB program run in 16 GB
- Evaluate the maximum acceptable page-fault rate for a 10% slow-down
- Explain how the number of frames per process should be decided (equal and proportional allocation)
- Explain how the OS detects and stops thrashing

Experimental Investigation

You must:

- Write a program that touches memory in sequential and random order and count its page faults
- Measure the run time as the memory used grows beyond the free RAM (use a virtual machine or a memory limit)
- Show copy-on-write with fork() by measuring memory before and after writing
- Plot the page faults against the number of frames for a reference string

Conclusion

Is buying more RAM or improving the program's locality the better solution for the radiologist? Justify.

Alternative Concept and Complexity Analysis

If the radiologist's program were run on a machine with 32 GB of RAM instead of relying on demand paging, what would the solution be? Evaluate the effective access time in both cases for the measured page-fault rate, and the cost of extra memory against the cost of the slower run.

Q5 – Page-Replacement Techniques — 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Choosing a Policy for a Web Server

A web server keeps frequently used pages of data in memory. When memory is full, the OS must choose which page to remove, and a poor choice causes slow responses.

Implementation Tasks

You must:

- Implement FIFO, LRU and Optimal page replacement
- Count the page faults for a given reference string with 3 and 4 frames
- Demonstrate Belady's anomaly with FIFO
- Explain the clock (second-chance) algorithm as a practical approximation of LRU

Evaluation

You must:

- Explain why Optimal is the best possible but cannot be implemented
- Explain why LRU never suffers from Belady's anomaly (the stack property)
- Evaluate the hardware support LRU needs and how the clock algorithm avoids it
- Compare the algorithms for a workload with strong locality

Experimental Investigation

You must:

- Generate reference strings with strong locality and with random access, and count the faults of each algorithm for 3 to 10 frames
- Plot the faults against the number of frames for each algorithm
- Measure the running time of each algorithm for 100,000 references
- Verify your program's results with a hand calculation for one string

Engineering Decision

Which page-replacement policy would you recommend for the web server? Use your measurements.

Complete the Code

Complete TODO 1 below to finish LRU, and execute the program.

```c
#include <stdio.h>

#define MAXF 8

int find(const int f[], int n, int page)
{
    for (int i = 0; i < n; i++) if (f[i] == page) return i;
    return -1;
}

int fifo(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, next = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        if (find(f, used, ref[i]) >= 0) continue;
        faults++;
        if (used < frames) f[used++] = ref[i];
        else { f[next] = ref[i]; next = (next + 1) % frames; }
    }
    return faults;
}

int lru(const int ref[], int len, int frames)
{
    int f[MAXF], last[MAXF], used = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        int k = find(f, used, ref[i]);
        if (k >= 0) { last[k] = i; continue; }
        faults++;
        // TODO 1: If a frame is free use it; otherwise replace the frame whose last use
        //   (last[]) is the oldest. Record last[] = i for the frame used

    }
    return faults;
}

int optimal(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        if (find(f, used, ref[i]) >= 0) continue;
        faults++;
        if (used < frames) { f[used++] = ref[i]; continue; }
        int victim = 0, farthest = -1;
        for (int j = 0; j < frames; j++) {
            int next = len;                  /* never used again */
            for (int t = i + 1; t < len; t++) if (ref[t] == f[j]) { next = t; break; }
            if (next > farthest) { farthest = next; victim = j; }
        }
        f[victim] = ref[i];
    }
    return faults;
}

int main(void)
{
    int ref[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};
    int len = 20;
    for (int fr = 3; fr <= 4; fr++)
        printf("%d frames: FIFO %d, LRU %d, Optimal %d\n", fr, fifo(ref, len, fr),
               lru(ref, len, fr), optimal(ref, len, fr));
    int b[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
    printf("Belady string, FIFO: 3 frames %d faults, 4 frames %d faults\n",
           fifo(b, 12, 3), fifo(b, 12, 4));
    return 0;
}
```

Expected output:

```
3 frames: FIFO 15, LRU 12, Optimal 9
4 frames: FIFO 10, LRU 8, Optimal 8
Belady string, FIFO: 3 frames 9 faults, 4 frames 10 faults
```

Explain why FIFO has more faults with 4 frames than with 3 for the second reference string, and why LRU cannot behave like this.

Alternative Concept and Complexity Analysis

If the clock (second-chance) algorithm were used instead of exact LRU, what would the solution be? Evaluate the time per reference (updating a time stamp or a list on every access versus setting one bit), the extra space per frame, and how close its page faults come to LRU in your simulation.

Q6 – Disk Management **4 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Storage for a Hospital Records Server

The hospital records server must start reliably, survive the failure of a disk and have enough swap space for peak load.

Implementation Tasks

You must:

- Explain low-level formatting, partitioning and logical formatting
- Explain the boot block and the boot process
- Explain how bad blocks are handled
- Decide the size and location of the swap space

Evaluation

You must:

- Compare RAID 0, 1, 5 and 10 for reliability, capacity and speed
- Choose the RAID level for the records server
- Evaluate swap on a separate disk versus a swap file
- Explain how disk partitions are viewed on Linux (lsblk, fdisk -l)

Experimental Investigation

You must:

- Show the partitions and swap of your Linux system
- Calculate the usable capacity of 6 disks of 4 TB at each RAID level
- Calculate the probability of data loss for RAID 1 and RAID 5 given a disk failure rate
- Measure the read speed of a file on SSD and hard disk if both are available

Conclusion

Which storage configuration would you recommend for the records server, and why?

Alternative Concept and Complexity Analysis

If the records server used RAID 6 instead of RAID 5 (or RAID 10), what would the solution be? Evaluate the usable capacity of 6 disks of 4 TB, the time to write a block (the extra parity calculation), and the number of disk failures that can be survived.

Q7 – Design a Memory-Management Plan from First Principles — 10 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Memory Management for an Edge Device in a Smart Factory

A smart factory installs edge devices with 2 GB of RAM and no swap disk. Each device runs a real-time control task that must never be delayed by a page fault, a machine-learning task that needs 1.5 GB, and a logging task. The vendor wants to add more tasks in the future.

You are not given the design. You must develop it from the requirements.

Stage 1 – Interpret the Problem | 2 Marks

- Identify the memory needs and timing constraints of each task
- Identify which tasks can tolerate page faults and which cannot
- State the requirements as measurable statements

Stage 2 – Compare Candidate Mechanisms | 3 Marks

- Compare contiguous allocation, paging, segmentation and demand paging for each task
- Decide how memory is locked for the real-time task (for example mlock())
- Present a comparison table with justification

Stage 3 – Implement and Verify | 3 Marks

- Write a program that locks the memory of a time-critical loop and measure its worst-case delay with and without locking
- Simulate the memory use of the three tasks with a page-replacement simulator
- Verify that the real-time task is never delayed

Stage 4 – Defend the Design | 2 Marks

- Justify your plan using your measurements
- Explain what changes if a fourth task of 500 MB is added
- Identify one remaining limitation

Alternative Concept and Complexity Analysis

If the edge device used a real-time operating system with no virtual memory instead of Linux with locked memory for the control task, what would the solution be? Evaluate the worst-case delay of the control task, the memory available to the machine-learning task, and the effort of adding new tasks.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Linking, loading and address binding	K5 – Evaluate	6
Q2	Paging	K5 – Evaluate	8
Q3	Segmentation	K5 – Evaluate	6
Q4	Virtual memory and demand paging	K5 – Evaluate	8
Q5	Page-replacement techniques	K5 – Evaluate	8
Q6	Disk management	K5 – Evaluate	4
Q7	Memory-management plan from first principles	K5 – Evaluate	10
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your report containing the required evidence for Q1–Q7
- C source code for all programs
- Address-translation workings, graphs, measurements and command outputs

Assessment Verification

Students may be asked to explain, execute or modify any part of their design, code, calculations or results during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

Q1. Linking, Loading and Address Binding (6 Marks)

1. Static Linking and Dynamic Linking

Static linking combines the required library code with the object files when the executable is built. The resulting executable contains the library routines it needs and is generally larger. Updating a library usually requires the application to be linked and distributed again. Dynamic linking keeps shared-library code separate from the executable. The required library is loaded by the dynamic loader when the program starts or when it is requested. This generally reduces executable size and allows multiple processes to share library code in physical memory.

Feature	Static linking	Dynamic linking
Executable size	Usually larger	Usually smaller
Library update	Normally rebuild/relink the application	Can update a compatible shared library separately
Memory use	Library code may be duplicated across executables	Read-only library pages can be shared
Startup	No shared-library resolution for statically linked code	Loader must locate and map required libraries
Dependency	Fewer external runtime library dependencies	Requires compatible shared libraries at runtime

2. Linker and Loader

The linker combines object files, resolves external symbols and incorporates or records library dependencies to create an executable. The loader prepares the executable for execution: it maps program segments into memory, loads or maps required shared libraries, performs necessary relocations, creates the process stack and transfers control to the program entry point.

3. Address Binding

- Compile-time binding: If the memory location is known in advance, the compiler or assembler can generate absolute addresses. Moving the program may require recompilation.
- Load-time binding: The compiler generates relocatable code and the final addresses are fixed when the program is loaded into memory.
- Execution-time binding: Address translation occurs while the program runs, allowing a process to move in physical memory. Hardware support such as a memory-management unit is required.

4. Position-Independent Code

Position-independent code (PIC) can execute correctly regardless of the memory address at which it is loaded. It uses position-independent addressing methods rather than depending on fixed absolute addresses. PIC is important for shared libraries because the same library code can be mapped at different virtual addresses in different processes.

5. Evaluation for the Banking Application

Dynamic linking is generally preferable for the bank because a security fix to a compatible encryption library can be deployed without rebuilding every branch executable. Shared read-only library pages can also be shared by multiple running processes, reducing physical-memory use. The bank must manage library versions and ensure that the required library is available and compatible. Static linking may be appropriate where a self-contained executable is more important than easy library updates.

6. Run-time Loading with dlopen()

With run-time loading, the application calls dlopen() only when encryption is required. It then uses dlsym() to obtain the address of the required function and calls dlclose() when the library is no longer needed. This can reduce startup work and avoid loading the encryption library when encryption is not used. The application must handle errors such as a missing library, an unavailable symbol or an incompatible library. The extra lookup and loading work occurs when the feature is first requested.

7. Experimental Investigation – Theory

Executable sizes can be compared using file-size information, and shared-library

dependencies can be listed with ldd. The process memory map in /proc/<PID>/maps shows mapped executable and library regions. Startup time and memory use should be measured repeatedly under the same conditions. Two program instances can be inspected through their memory maps and memory-accounting information to identify shared file-backed pages. If the shared library is removed or made unavailable, a dynamically linked program normally fails to start; a program using dlopen() can detect the failure and report it when the library is requested. Actual timings and memory measurements must be recorded from the student's own system.

8. Complexity Analysis and Conclusion

Ordinary symbol resolution and relocation depend on the number of symbols and relocations; their exact cost depends on the linker and executable. At run time, a dynamic loader performs library lookup and relocation work, while dlopen() adds lookup and loading work at the point of use. Static and dynamic linking do not change the asymptotic complexity of the bank's encryption algorithm. For this banking scenario, dynamic linking is the best general choice because security-library updates can be distributed more easily and library pages can be shared. Run-time loading is useful if encryption is optional or rarely used.

Q2. Paging (8 Marks)

1. External Fragmentation

External fragmentation occurs when free memory is split into many small, non-contiguous holes. The total free memory may be sufficient for a process, but no single free block is large enough to hold it. This problem occurs with variable-sized contiguous allocation.

2. Paging and Address Translation

Paging divides a process's logical address space into fixed-size pages and physical memory into equal-size frames. A page table maps each virtual page number to a physical frame number. A logical address consists of a page number and an offset. The offset is unchanged during translation.

Page number = floor(Logical address / Page size). Offset = Logical address mod Page size. Physical address = (Frame number × Page size) + Offset.

For example, with a page size of 1024 bytes, logical address 1500 belongs to page 1 and has offset 476. If page 1 is stored in frame 2, the physical address is (2 × 1024) + 476 = 2524.

3. Page-Table Size

Assume a 4 KB page size and a 4-byte page-table entry, with a fully populated single-level page table. A 32-bit address space contains 2^32 / 2^12 = 2^20 pages. Its page table requires 2^20 × 4 bytes = 4 MB. A 48-bit address space contains 2^48 / 2^12 = 2^36 pages. Its page table requires 2^36 × 4 bytes = 256 GB. These are theoretical single-level sizes; practical systems use multi-level or other page-table structures so that unused address-space regions do not require fully populated tables.

4. Effective Access Time and TLB

A Translation Lookaside Buffer (TLB) is a small, fast cache that stores recently used page-table entries. Without a TLB, a simple single-level page-table access requires one memory access to read the page-table entry and another to access the data. If memory access time is 100 ns, the effective time is approximately 200 ns.

Assuming a TLB lookup takes 10 ns, a TLB hit takes 10 + 100 = 110 ns. A TLB miss takes 10 + 100 + 100 = 210 ns. Therefore, EAT = h(110) + (1 − h)(210), where h is the TLB hit ratio.

TLB hit ratio	Effective access time
80%	130 ns
95%	115 ns
99%	111 ns

5. How Paging Reduces Fragmentation and Page-Size Evaluation

Paging removes external fragmentation because any free frame can hold any page; a process does not need one continuous physical-memory block. Internal fragmentation can remain because the final page of a process may not be completely used.

Page size	Advantages	Disadvantages
1 KB	Small amount of unused space in the final page	More pages and larger page tables; more mapping overhead

4 KB	Common balance between mapping overhead and wasted space	Can still produce internal fragmentation
2 MB	Fewer page-table entries and useful for large contiguous working sets	Potentially large internal fragmentation; less flexible for small allocations

For a general-purpose college server, 4 KB pages are a reasonable default because they balance memory waste and page-table overhead. Large pages may be useful for selected large workloads.

6. Page-Table Structures

- Hierarchical (multi-level) page table: Divides the page table into levels. Only the portions needed for mapped address-space regions need to be created, reducing page-table memory use for sparse address spaces. Translation may require walking multiple levels after a TLB miss.
- Hashed page table: Uses a hash function on the virtual page number to locate a page-table entry. It is useful for large address spaces; collisions must be handled.
- Inverted page table: Maintains an entry for each physical frame rather than one entry for every virtual page of every process. Its size is related to physical memory, but lookup may require hashing or searching.

7. Protection and Shared Pages

Page-table entries contain protection information such as read, write and execute permissions. The processor checks these permissions during memory access and raises a fault for an illegal operation. Shared pages are implemented by mapping the same physical frame into the address spaces of multiple processes. Read-only code pages can be safely shared, while writable data is normally kept private or managed using copy-on-write.

8. Experimental Investigation and Complexity

For each process, internal fragmentation is the unused space between the process's requested size and the page-rounded allocation. For a page size P and process size S, allocated space is ceil(S/P) × P, and internal fragmentation is allocated space − S. The

average for 100 processes is the sum of their internal fragmentation divided by 100. Use the same 100 process sizes when comparing 1 KB, 4 KB and 2 MB pages.

The Linux page size can be obtained from the system configuration, and a program's minor and major page faults can be measured with system resource-reporting tools. Address tests should include the first byte of a page, the last byte of a page and an address beyond the valid page range. A multi-level page-table walk has a bounded number of levels; a TLB hit avoids that walk. An inverted table uses space proportional to physical frames, while a per-process page table is related to the process virtual address space.

9. Conclusion and Alternative: Inverted Page Table

Paging is suitable for the college server because it allows programs of different sizes to occupy non-contiguous physical frames and avoids external fragmentation. An inverted page table would reduce table space for systems with very large virtual address spaces because it stores one entry per physical frame. Address translation would use a hash lookup to find the virtual-page and process identifier, rather than walking a per-process multi-level page table. The hash approach can be efficient but requires collision handling and may have variable lookup time.

Q3. Segmentation (6 Marks)

1. Meaning and Address Translation

Segmentation divides a program into logical, variable-sized units such as code, data, heap and stack. A logical address contains a segment number and an offset. The segment table stores a base address and a limit for each segment. If the offset is less than the segment limit, the physical address is base + offset. If the offset is equal to or greater than the limit, the reference is invalid and a protection fault is raised.

For example, if a data segment has base 2000 and limit 300, offset 100 translates to physical address 2100. Offset 350 is invalid because it exceeds the segment limit.

2. Segmentation with Paging

Segmentation with paging first identifies the logical segment. Each segment has an associated page table, and the segment's offset is divided into a page number and page offset. The page table maps the page to a physical frame, and the page offset identifies

the byte within that frame. This combines the logical organization and protection of segmentation with the non-contiguous allocation of paging.

3. Linux Process Memory Regions

A Linux process memory map can show regions for the executable, shared libraries, heap, stack and memory-mapped files. These regions illustrate how a running process uses separate virtual-memory areas. Modern Linux on common x86-64 systems relies mainly on paging for virtual-memory management; the visible mappings should not be confused with the older use of hardware segmentation as the primary memory-management method.

4. Advantages and Disadvantages

- Segmentation matches the programmer's logical view of a program and allows separate protection for code, data and stack.
- Code segments can be shared between processes, and individual segments can have different access permissions.
- Because segments have variable sizes, free memory can become divided into external fragments.
- Managing segment tables and finding suitable free blocks adds memory-management complexity.

Feature	**Pure paging**	**Pure segmentation**	**Segmented paging**
Division	Fixed-size pages	Variable-size logical segments	Logical segments divided into fixed-size pages
Fragmentation	Internal	External	Internal within pages
Allocation	Any available frame	Requires suitable segment space	Pages of each segment can occupy separate frames
Protection	Page-level	Segment-level	Segment and page-level
Sharing	Share selected pages	Share logical segments	Share pages belonging to segments

5. Experimental Investigation

Eight (segment, offset) pairs should be checked against the segment table. Valid offsets are translated by adding the base; offsets at or beyond the limit must be rejected. A segmentation fault occurs when a process performs an invalid memory access, such as writing through a null pointer. The processor detects the protection violation and the operating system delivers a fault signal, commonly terminating the process if it is not handled.

For 20 variable-sized segments, external fragmentation can be measured as total free memory minus the size of the largest free block. The table-space comparison depends on the number of segments, pages and physical frames. Pure paging uses page-table entries for virtual pages; segmentation uses entries for logical segments; segmented paging uses segment entries plus page tables for the pages within each segment.

6. Conclusion and Alternative: Pure Paging

Modern operating systems rely mainly on paging because fixed-size frames eliminate external fragmentation and support flexible allocation, protection and sharing. If pure paging replaced segmentation, code, data and stack would all be represented as pages. Internal fragmentation could occur in the final page, but external fragmentation would be avoided. Page tables may be larger than a simple segment table, although multi-level tables reduce unused entries. Address translation uses a TLB and, on a miss, a page-table walk. Code sharing and protection remain possible through page-table permissions and shared physical frames, but the logical separation of code, data and stack must be represented by separate virtual-memory regions.

Q4. Virtual Memory and Demand Paging (8 Marks)

1. Virtual Memory and Demand Paging

Virtual memory gives a process a large logical address space and allows only the currently needed pages to reside in physical RAM. Demand paging loads a page into RAM only when the process first references it. Pages not currently resident can remain in backing storage. This allows a program whose virtual-memory requirement is larger than physical RAM to run, provided its active working set and system overhead can be

accommodated and backing storage is available.

2. Page-Fault Handling

1. The CPU references a virtual page whose page-table entry indicates that it is not present in RAM.
2. The hardware raises a page-fault exception and transfers control to the operating system.
3. The operating system checks whether the memory reference is valid and permitted.
4. If valid, the OS finds a free frame or selects a page for replacement.
5. The required page is read from backing storage into the selected frame.
6. The page table and TLB are updated, and the interrupted instruction is restarted.

3. Effective Access Time

Given memory access time = 100 ns and page-fault service time = 8 ms = 8,000,000 ns. Using EAT = (1 − p) × 100 + p × (8,000,000 + 100), where p is the page-fault probability:

Fault rate	**p**	**Effective access time**
1 in 1,000	0.001	8,099.9 ns
1 in 100,000	0.00001	179.999 ns (approximately 180 ns)
1 in 1,000,000	0.000001	107.9999 ns (approximately 108 ns)

4. Copy-on-Write

Copy-on-write (COW) allows a parent and child process created by fork() to initially share the same physical pages. The pages are marked so that a write causes a protection fault. The operating system then creates a private copy for the process that writes, updates its page table and resumes execution. This makes fork() faster because the system does not immediately copy all of the parent's memory.

5. Thrashing and Working Set

Thrashing occurs when a system spends most of its time servicing page faults and transferring pages between RAM and storage instead of executing useful instructions. It can be caused by too many active processes or insufficient frames for their working sets.

The working-set model estimates the set of pages a process has referenced during a recent time window. Giving a process enough frames for its working set helps reduce page faults. If the total working-set demand is greater than available frames, the OS can reduce the degree of multiprogramming by suspending or swapping out some processes.

6. Running a 24 GB Program on a 16 GB Laptop

Demand paging allows the 24 GB medical-imaging program to execute on a laptop with 16 GB RAM because all 24 GB do not need to be resident at the same time. The OS loads pages as they are referenced and can move less-used pages to backing storage. Performance may become poor if the active working set is larger than available RAM, causing frequent page faults. Improving locality reduces the number of pages needed at one time.

7. Maximum Page-Fault Rate for 10% Slowdown

A 10% slowdown over a 100 ns memory access means a maximum EAT of 110 ns. Using EAT = 100 + p × 8,000,000, the condition is 100 + 8,000,000p ≤ 110. Therefore p ≤ 10/8,000,000 = 0.00000125, which is approximately one fault per 800,000 memory references. This calculation assumes the stated page-fault service time and ignores other overheads.

8. Frame Allocation and Thrashing Control

Equal allocation gives each process the same number of frames, regardless of its size. It is simple but may not suit processes with very different memory requirements. Proportional allocation gives frames according to each process's size or memory demand. The OS can monitor page-fault frequency and working-set demand. If a process faults too frequently, it may receive more frames; if the system cannot supply them, some processes may be suspended to reduce memory pressure.

9. Experimental Investigation and Conclusion

Sequential memory access generally has stronger locality than random access and may generate fewer major page faults once pages are loaded. A memory-touching experiment should increase the allocated working set beyond available free RAM and record page faults and execution time. A fork() experiment can compare memory use before and after the child writes to shared pages. A page-replacement simulation can plot page faults against frame count. Actual timing and fault counts must be measured on the test system.

For the radiologist, improving program locality is often the first cost-effective step because it reduces unnecessary page transfers. However, if the program's active working set genuinely requires more than 16 GB, adding RAM is likely to provide a substantial performance improvement. The best decision depends on measured working-set size and the cost of slower execution.

10. Alternative: 32 GB RAM

With 32 GB RAM, the 24 GB program can potentially keep its working data in physical memory, reducing page faults caused by memory pressure. Its access time approaches ordinary memory access time when the required pages are resident. With 16 GB, the effective access time depends on the measured page-fault rate and can become much higher because an 8 ms page fault is extremely costly compared with a 100 ns memory access. The extra RAM has a purchase cost, while demand paging has a performance cost. If the workload repeatedly uses more than 16 GB, additional RAM may be justified.

Q5. Page-Replacement Techniques (8 Marks)

1. Meaning of Page Replacement

When a page fault occurs and all physical frames are occupied, the operating system must select a resident page to remove. The selected page is called the victim page. A good replacement policy reduces page faults while keeping implementation overhead low.

2. FIFO, LRU and Optimal

- FIFO (First-In, First-Out): Replaces the page that has been in memory the longest. It is simple to implement using a queue, but it may remove a frequently used page and can suffer from Belady's anomaly.
- LRU (Least Recently Used): Replaces the page that has not been used for the longest time in the past. It uses recent history as an estimate of future use and generally performs well when programs have locality.
- Optimal: Replaces the page whose next use is farthest in the future, or that will never be used again. It gives the minimum possible number of page faults for a known reference string, but cannot be implemented exactly online because future references are unknown.

3. Belady's Anomaly and Stack Property

Belady's anomaly is the situation in which increasing the number of page frames causes FIFO to produce more page faults. FIFO does not have the stack property, so the pages present with a smaller number of frames are not necessarily a subset of those present with more frames. LRU has the stack property: the pages held with n frames form a subset of those held with n+1 frames. Therefore, increasing the number of frames cannot increase LRU page faults.

For the assignment's Belady reference string, FIFO produces 9 faults with 3 frames and 10 faults with 4 frames. This demonstrates the anomaly. The assignment's supplied expected results for its main reference string should be checked against the exact reference string and completed code before reporting measured values.

4. Clock (Second-Chance) Algorithm

The Clock algorithm is a practical approximation of LRU. Frames are arranged in a circular list and a pointer identifies the next candidate. Each frame has a reference bit. When a page is accessed, its reference bit is set to 1. During replacement, a page with reference bit 1 is given a second chance: the bit is cleared and the pointer advances. A page with bit 0 is selected for replacement. This avoids maintaining a precise timestamp or ordering structure on every access.

5. Evaluation of Algorithms

Algorithm	Strength	Limitation
FIFO	Simple queue and low bookkeeping cost	May remove frequently used pages; Belady's anomaly
LRU	Uses recency and performs well with locality	Exact implementation needs access-history tracking or hardware support
Optimal	Theoretical minimum page faults	Requires knowledge of future references
Clock	Low overhead; one reference bit per frame	Approximation may have more faults than exact LRU

Exact LRU can be implemented using timestamps or a list that is updated on each access. These updates add time and bookkeeping overhead. Hardware reference bits make Clock

efficient because accesses mainly set a bit, while the replacement scan clears bits and advances a pointer. Clock generally approaches LRU when recently used pages are likely to be used again, but its actual fault count depends on the reference pattern.

6. Workload Locality and Experimental Investigation

A workload with strong locality repeatedly accesses a relatively small group of pages. LRU and Clock tend to perform well because recently used pages are likely to be referenced again. FIFO may evict a useful page simply because it is old. Random access has weaker locality and may cause more page faults for all policies. The assignment requires reference strings to be generated for both patterns, with page faults counted for 3 to 10 frames and results plotted against frame count. Running time for 100,000 references should also be measured. These are experimental results and must be obtained by executing the program.

7. Complexity Analysis and Engineering Decision

For a simple implementation that searches all frames on each reference, FIFO, LRU and Optimal can each take O(F) time per reference, where F is the number of frames. Over N references, this is O(NF). FIFO can be implemented with O(1) queue updates per reference. LRU can achieve O(1) updates with a suitable hash table and doubly linked list, but requires O(F) metadata. A straightforward Optimal implementation searches the remaining reference string for each resident page and can take O(NF) per reference in the worst case; it is mainly a benchmark. Clock uses O(1) reference-bit updates and may scan up to O(F) frames during replacement, with O(F) space.

For a web server, LRU or Clock is generally more practical than Optimal. LRU is suitable when strong locality makes recency a good predictor, while Clock offers lower bookkeeping overhead and is widely useful as an approximation. The final choice should be supported by the measured page faults and running time for the server-like workload.

Q6. Disk Management (4 Marks)

1. Disk Formatting

- Low-level formatting: Organizes a disk into sectors and establishes the structures needed for sector-level storage. Modern drives are generally factory low-level formatted.

- Partitioning: Divides a physical disk into logical regions called partitions. Each partition can be used for a file system, swap or another purpose.
- Logical formatting: Creates a file system, such as ext4 or NTFS, within a partition so that files and directories can be stored.

2. Boot Block and Boot Process

A boot block or bootloader-related boot area contains code or data used to begin loading the operating system. In a typical modern system, firmware (BIOS or UEFI) initializes hardware and selects a boot device. The bootloader is then executed, loads the operating-system kernel and transfers control to it. The kernel initializes memory management and device drivers, mounts the required file systems and starts the initial system services.

3. Bad Blocks

Bad blocks are disk sectors that cannot reliably store or return data. Drives may detect defects and remap affected sectors to spare sectors. The operating system and file system can also identify unusable blocks and avoid allocating them. Error-correcting codes help detect and correct certain errors. Monitoring disk health and maintaining backups are important because remapping cannot guarantee that a failing disk will remain reliable.

4. Swap Space

Swap is disk space that can back pages moved out of RAM. Its size should be based on the operating system, workload, memory pressure and any requirement for hibernation; there is no universal fixed ratio. Swap can be a dedicated partition or a swap file. A separate disk may provide independent I/O if it is not competing with the main workload, while a swap file is easier to create, resize and manage. Swap is much slower than RAM and should not be treated as a substitute for sufficient physical memory.

5. RAID Comparison and Capacity for Six 4 TB Disks

RAID level	**Usable capacity**	**Fault tolerance**	**General performance**
RAID 0	24 TB	No disk-failure protection	High throughput; a single disk failure can lose the array
RAID 1	Depends on mirror	Mirroring protects	Good reads; writes

	layout; three mirrored pairs provide 12 TB	against a disk failure in each mirror pair, subject to layout	go to mirrors
RAID 5	(6−1) × 4 = 20 TB	Can survive one disk failure	Good reads; parity adds write overhead
RAID 10	(6/2) × 4 = 12 TB	Can survive multiple failures if they are not in the same mirror pair	Good read and write performance

RAID 0 stripes data for performance but has no redundancy. RAID 1 mirrors data. RAID 5 distributes parity and can recover from one failed disk. RAID 10 combines mirroring and striping, providing good performance and redundancy but using half the raw capacity.

6. Recommended RAID for Hospital Records

RAID 10 is a strong choice when the hospital prioritizes write performance, predictable recovery and redundancy. RAID 6 is also worth considering where surviving two disk failures is more important than write performance. RAID is not a backup: separate, tested backups and a recovery plan are still required.

7. Probability of Data Loss

The probability depends on the assumed disk-failure probability and whether failures are independent. For a two-disk mirror with independent per-disk failure probability p over the same period, data loss requires both disks in that mirror pair to fail, giving p^2. For a six-disk RAID 5, data loss occurs when two or more disks fail. Under independent identical failure probability p, P(loss) = 1 − (1−p)^6 − 6p(1−p)^5. These simplified models do not include correlated failures, rebuild time, latent sector errors or controller faults.

8. Linux Disk Inspection and Experimental Investigation

Linux disk and partition information can be viewed with tools such as lsblk and fdisk; swap devices can be viewed with swapon. The student should record the actual partitions and swap configuration from the test system. Read performance should be measured on

an SSD and HDD if both are available, using the same file size, cache conditions and test method. The measured values should be reported rather than assumed.

9. Alternative: RAID 6

For six 4 TB disks, RAID 6 usable capacity is (6−2) × 4 TB = 16 TB. RAID 6 stores two independent parity blocks and can survive any two disk failures in the array. Writes require additional parity calculation and updates, so small-write performance is generally lower than RAID 10. RAID 6 is preferable when double-disk-failure tolerance is a priority; RAID 10 is preferable when performance and simpler rebuild behaviour are more important.

Q7. Design a Memory-Management Plan from First Principles (10 Marks)

Stage 1 – Interpret the Problem

The edge device has 2 GB RAM and no swap disk. It runs a real-time control task, a machine-learning task requiring 1.5 GB and a logging task. The control task has the strictest timing requirement: it must not be delayed by a page fault. The machine-learning task needs a large memory allocation and may tolerate some delay only if its application requirements permit it. Logging is usually less time-critical and can tolerate buffering or delayed writes.

Measurable requirements include: keep the real-time working set resident in RAM; prevent page faults in the control task's critical path; keep total committed and resident memory within the device's physical-memory limits; and ensure that logging does not consume memory reserved for control. The exact memory allocation for control and logging must be measured because the problem does not specify their sizes.

Stage 2 – Compare Candidate Mechanisms

Mechanism	Benefits	Limitations for this device
Contiguous allocation	Simple and predictable for fixed regions	Can cause external fragmentation and makes flexible growth difficult
Paging	Uses fixed-size frames and avoids external fragmentation	Page faults can cause unpredictable delays

Segmentation	Separates code, data and stack logically and supports per-region protection	Variable-sized regions can cause external fragmentation
Demand paging	Loads pages only when needed and can reduce initial RAM use	A page fault can take far longer than a real-time deadline

The real-time task's code, stack and required data should be allocated and prefaulted before entering its critical operation, then locked in RAM using a mechanism such as mlock(), subject to operating-system limits and privileges. Memory locking reduces the risk that those pages will be reclaimed or swapped. The machine-learning and logging tasks can use the remaining memory, but their allocations must be controlled so that they cannot force the real-time task's working set out of memory.

Stage 3 – Implementation and Verification

A verification program should allocate the real-time working memory, touch every page before timing begins, and compare repeated timing measurements with and without memory locking. The maximum observed delay should be reported, but it is not a formal worst-case execution-time guarantee. The test should be repeated under representative CPU and memory load. Page-fault counters should be checked to confirm that the critical loop does not incur faults after initialization.

A memory-use simulation should represent the three tasks and their working sets, then apply a page-replacement policy to non-critical pages. Verification should confirm that the real-time task's pages remain resident and that memory demand does not exceed the 2 GB physical limit. Since the machine-learning task alone needs 1.5 GB, only 0.5 GB remains in the nominal 2 GB before accounting for the operating system, control task, logging task and other overhead. Therefore, the device may need reduced ML memory use or more RAM.

Stage 4 – Defend the Design

The design reserves and locks the real-time task's working set, while managing the machine-learning and logging tasks within the remaining memory. This is justified because page faults can introduce millisecond-scale delays, which are unacceptable in a strict real-time control path. Measurements must demonstrate that the critical path has no

page faults during the test and that its latency meets the specified deadline.

Adding a fourth task that requires 500 MB increases total memory demand by 500 MB. The device has only 2 GB RAM, and the machine-learning task already requires 1.5 GB. The new task cannot simply be added without reducing other allocations, limiting its working set, scheduling it at a different time, or upgrading the device's RAM. One remaining limitation is that memory locking alone does not guarantee real-time behaviour: CPU scheduling, interrupts, device I/O and kernel configuration also affect latency.

Alternative Concept: Real-Time Operating System Without Virtual Memory

With an RTOS that does not use virtual-memory paging, memory can be statically partitioned or allocated from fixed pools. The real-time control task can receive a reserved region, so it does not experience virtual-memory page faults. This can improve predictability and simplify worst-case timing analysis. However, the machine-learning task has less flexibility and must fit within the remaining physical memory. Adding new tasks requires careful memory budgeting, static allocation and retesting of timing behaviour. The RTOS approach is attractive for strict real-time control, while Linux with locked memory may offer broader software support and flexibility.

Overall Conclusion

Memory management techniques are selected according to system requirements. Dynamic linking supports easier software updates; paging supports flexible memory allocation; segmentation provides logical protection; demand paging enables large virtual address spaces; page-replacement algorithms manage limited frames; and RAID and swap support storage reliability and memory pressure. The best design is the one that satisfies performance, reliability, memory and timing requirements while being supported by actual measurements.